

Flujo ELK mixto: ingesta en lote y en near real- time sobre el sistema de banca privada del Ecuador

Informe técnico · Taller de la semana 2

Arquitectura: Elasticsearch 8.15.3 · Logstash 8.15.3 · Kibana 8.15.3 · Filebeat 8.15.3, sobre Docker Compose

Fuentes integradas: cinco — tres en lote y dos en near real-time · **Fecha:** 03/09/2026

1. Objeto y alcance

Se diseña e implementa un flujo de ingesta sobre la pila ELK que combina los dos esquemas de procesamiento en una sola arquitectura: un **esquema en lote** que traslada a Elasticsearch el Data Warehouse dimensional construido en el taller anterior, y un **esquema near real-time** que ingiere un flujo continuo de transacciones con latencia de segundos. En total se integran **cinco fuentes de datos** sobre tres tecnologías de origen distintas — PostgreSQL, MySQL y ficheros planos — y dos mecanismos de transporte opuestos: uno que empuja el dato y otro que lo pregunta.

La totalidad de la infraestructura se declara en Docker Compose y se levanta con un único perfil, de modo que el flujo es reproducible en cualquier equipo sin instalar nada en el sistema anfitrión.

5

Fuentes integradas
(3 lote + 2 near real-time)

189.003

Documentos indexados
en Elasticsearch

1.099 ms

Latencia p95 del canal
near real-time

2,55 GiB

Memoria de los 8
contenedores en marcha

2. Arquitectura implementada

La arquitectura se organiza en dos carriles que comparten destino. El carril superior es el de lote: tres pipelines leen el almacén dimensional, los catálogos maestros y un fichero delimitado, y terminan cuando agotan su origen. El carril inferior es el near real-time: permanece a la escucha de forma indefinida y entrega cada evento en cuestión de segundos.

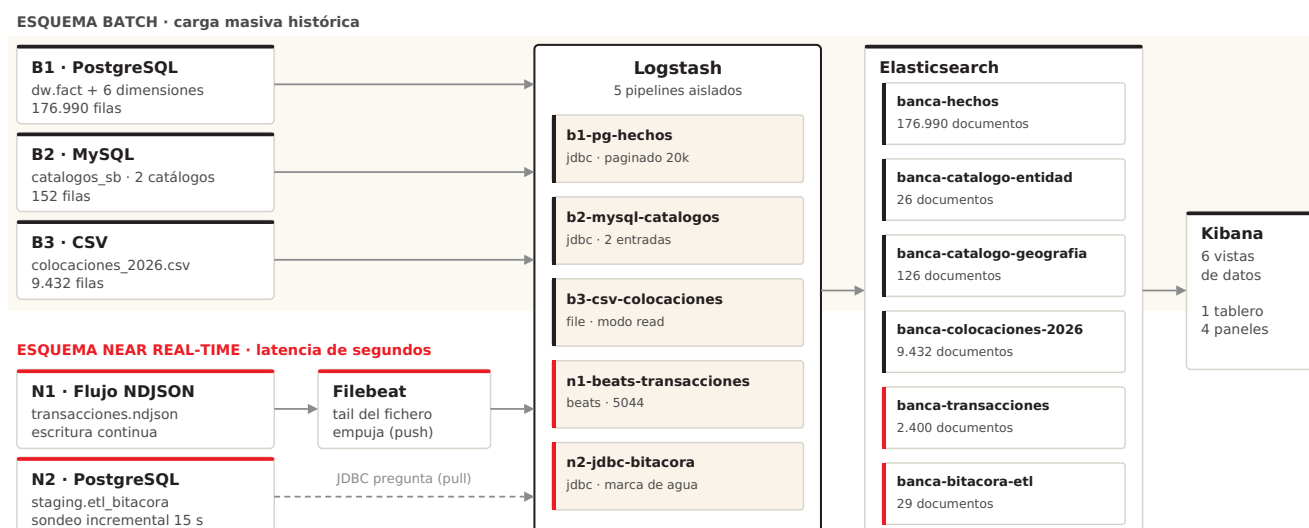


Figura 1. Flujo mixto de extremo a extremo. En rojo, los componentes del carril near real-time; la flecha discontinua representa el sondeo incremental, que pregunta en lugar de recibir.

2.1 Las cinco fuentes y su destino

Flujo	Origen	Esquema	Índice o flujo de datos	Documentos
B1	PostgreSQL · DW	Lote	banca-hechos	176.990
B2	MySQL · entidades	Lote	banca-catalogo-entidad	26
B2	MySQL · geografia	Lote	banca-catalogo-geografia	126
B3	CSV	Lote	banca-colocaciones-2026	9.432
N1	NDJSON → Filebeat	Near real-time	banca-transacciones	2.400
N2	PostgreSQL incremental	Near real-time	banca-bitacora-etl	29
Total				189.003

3. El carril en lote

Los tres pipelines en lote resuelven el traslado de un modelo relacional a un motor documental. El paso clave es la **desnormalizacion**: Elasticsearch no ejecuta uniones entre índices, de modo que la consulta de extracción une la tabla de hechos con sus seis dimensiones y produce un documento autocontenido con las veintiseis dimensiones de análisis y las ocho métricas de cada registro.

- **B1 — PostgreSQL.** Recupera los 176.990 hechos del almacén con paginación de 20.000 filas; sin ella, el controlador cargaría el resultado completo en la memoria de Logstash. La clave subrogada `saldo_sk` se usa como identificador del documento.
- **B2 — MySQL.** Dos entradas en un mismo pipeline, separadas por el campo `type` y encaminadas a índices distintos en la salida. El catálogo de entidades conserva las versiones de la dimensión lentamente cambiante, con una clave compuesta `cod_entidad-vN`.

- **B3 — CSV.** Lector en modo `read`, que procesa el fichero completo y termina, frente al modo `tail` que emplea el carril near real-time. Aplica la misma normalización de razones sociales que el flujo del taller anterior.

Conciliación con el origen. La suma de `saldo_total` agregada por Elasticsearch asciende a USD 2.926.925,37 millones, cifra idéntica a la que devuelve PostgreSQL sobre la tabla de hechos. El traslado no altera ningún importe.

4. El carril near real-time

El regulador publica boletines mensuales, por lo que no existe un flujo transaccional público que se pueda consumir en vivo. Para ejercitar el esquema se genera el evento que falta, pero **sin inventar el maestro**: las entidades, provincias y cantones se leen de los catálogos reales ya cargados en MySQL, de modo que el canal en vivo es consistente con el histórico y ambos se pueden cruzar en la misma consulta.

Se implementaron los dos mecanismos de ingesta continua, que son opuestos por diseño:

	N1 — empuje (push)	N2 — sondeo (pull)
Origen	Fichero NDJSON en crecimiento	Tabla <code>staging.etl_bitacora</code>
Transporte	Filebeat sigue el fichero y empuja a Logstash por el puerto 5044	Logstash consulta por JDBC cada 15 segundos
Control de avance	Registro de posición en el fichero	Marca de agua sobre <code>id_ejecucion</code> , guardada en disco
Latencia	p50 645 ms · p95 1.099 ms	Hasta 15 s, fijada por el intervalo de sondeo
Uso	Eventos de negocio	Observabilidad del propio proceso de carga

4.1 Enriquecimiento en vuelo

Es el punto donde los dos carriles se encuentran. Las transacciones no traen el grupo de tamaño de la entidad; el pipeline lo resuelve contra un diccionario materializado desde el mismo catálogo de MySQL que alimenta el flujo en lote, con lo que cada evento llega al índice ya clasificado. La resolución es **en memoria** y no por consulta: la sección 9 explica por qué la primera versión, que preguntaba a Elasticsearch una vez por evento, hubo que descartarla. Se evalúan además dos reglas de negocio en línea: se marca con `MONTO_ALTO` toda operación de USD 3.000 o más — 263 en esta corrida — y se etiquetan las 89 transacciones rechazadas.

Grupo de la entidad (resuelto en vuelo)	Transacciones	Reparto
PEQUEÑO	1.103	46,0 %
GRANDE	965	40,2 %
MEDIANO	332	13,8 %

5. Resultados medidos

La corrida que documenta este informe emitió 2.400 transacciones a lo largo de 123 segundos, es decir 19,6 eventos por segundo sostenidos, por un importe agregado de USD 2,28 millones.



Figura 2. Caudal de ingesta del canal near real-time. Cada barra agrupa diez segundos de emisión; la regularidad indica que el pipeline absorbe el flujo sin acumular retraso.

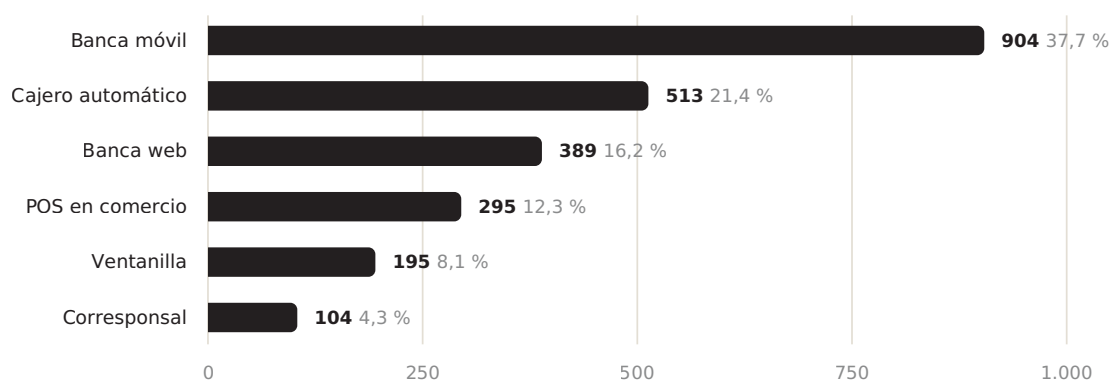


Figura 3. Reparto de las transacciones por canal de atención, calculado por Elasticsearch sobre el índice en vivo.

5.1 Latencia extremo a extremo

Cada evento lleva su propia medida: el pipeline resta la marca temporal que fija el origen del instante en que Logstash lo procesa.

Percentil	Latencia
p50	645 ms
p90	1.050 ms
p95	1.099 ms
p99	1.142 ms

5.2 Tipo de operacion

La distribución reproduce el perfil del gasto minorista, con predominio del consumo con tarjeta y del retiro en efectivo.

Tipo	Transacciones
Retiro	626
Consumo Tarjeta	616
Transferencia	451
Pago Servicio	416
Deposito	291

5.3 Reparto de la memoria

El equipo dedica 3,83 GiB a Docker, cifra inferior a la que reclaman los tres servicios de la pila con su configuración por defecto. La tabla recoge el consumo real de los ocho contenedores en marcha.

Contenedor	En uso (MB)	Límite fijado (MB)
elk_elasticsearch	1.067	1.399
elk_logstash	855	1.024
elk_kibana	452	800
elk_filebeat	95	150
etl_postgres	51	—
etl_mysql	43	—
etl_runtime	36	—
etl_adminer	6	—
Total	2.609	

6. Modelo de datos en Elasticsearch

Se registran cinco plantillas de índice antes de cualquier escritura. La plantilla fija el tipo de cada campo, la ausencia de replicas — obligada en un cluster de un solo nodo — y un analizador `es_sin_tildes` que aplica asciifolding, de modo que la búsqueda de «ATLANTIDA» encuentra también «ATLÁNTIDA»: los boletines del regulador mezclan ambas grafías.

El canal de transacciones no escribe en un índice corriente sino en un **flujo de datos** (data stream), la estructura que Elasticsearch reserva a series temporales de alta rotación. Solo admite escrituras de tipo `create` y delega en una política de ciclo de vida el relevo de índices y su eliminación.

Aspecto	Configuración
Flujo de datos	<code>banca-transacciones</code> , generación 1, 1 índice(s) de respaldo
Política de ciclo de vida	<code>banca-nrt</code> — relevo al alcanzar 1 GB o un día; eliminación a los 7 días
Refresco	1 s en el canal en vivo · 5 s en los índices de lote
Fragmentos	1 primario y 0 réplicas por índice

7. Explotación en Kibana

Los objetos de Kibana se crean por script y no a mano, para que el entregable sea reproducible: se declaran seis vistas de datos — una por flujo — y un tablero de cuatro paneles que confronta deliberadamente los dos esquemas en una misma pantalla. Los dos paneles superiores muestran el historico mensual del almacén; los dos inferiores, el caudal de transacciones por minuto y la latencia de ingesta p95, que solo tienen sentido en el carril en vivo.

8. Verificación

El script `elk/03_verificacion.sh` ejecuta ocho pruebas sobre el sistema en marcha.

Prueba	Resultado
Salud del clúster	Estado green, 34 fragmentos activos, 0 sin asignar
Conciliación de filas B1 (PostgreSQL ↔ Elasticsearch)	176.990 en ambos extremos
Conciliación de importes B1	USD 2.926.925,37 millones en ambos extremos
Carga de catálogos B2	26 entidades y 126 cantones
Carga del CSV B3	9.432 documentos, igual que las líneas del fichero
Canal near real-time N1	2.400 transacciones, p95 de 1.099 ms
Sondeo incremental N2	29 registros y marca de agua persistida en disco
Enriquecimiento entre carriles	0 transacciones sin correspondencia en el catálogo

9. Retos encontrados

Se documentan los 19 obstáculos que exigieron un cambio de diseño o de configuración. Todos se manifestaron durante la implementación y ninguno aparecía en la documentación consultada de forma evidente; varios eran silenciosos, que es lo que los hacía peligrosos.

Reto y síntoma observado	Resolución
1. Memoria: el equipo solo ofrece 3,8 GiB a Docker Elasticsearch, Logstash y Kibana reclaman en conjunto más de 4 GiB con su configuración por defecto, de modo que el stack no arrancaba junto a PostgreSQL y MySQL.	Se acotaron los heaps de la máquina virtual de Java (Elasticsearch 512 MB, Logstash 384 MB, Kibana 448 MB), se fijó un <code>mem_limit</code> por servicio y se desactivó el <code>performance_schema</code> de MySQL, que por sí solo liberó 400 MB. Los ocho contenedores conviven ahora en 2,55 GiB.
2. <code>path.config</code> anula <code>pipelines.yml</code> Logstash arrancó un único pipeline llamado <code>main</code> que concatenaba los cinco ficheros <code>.conf</code> : cada evento salía por las cinco salidas a la vez. El síntoma fue un índice de transacciones con 18.375 documentos antes de que el simulador se hubiera ejecutado, y documentos cuyo <code>_id</code> era el texto literal <code>%{saldo_sk}</code> porque la sustitución no aplicaba en el índice equivocado.	Se eliminó <code>path.config</code> de <code>logstash.yml</code> . Esa opción tiene prioridad sobre <code>pipelines.yml</code> y desactiva el aislamiento entre pipelines.
3. El borrado con comodín se rechaza en silencio <code>DELETE /banca-*</code> devuelve un error 400 porque el ajuste <code>action.destructive_requires_name</code> vale <code>true</code> por defecto en Elasticsearch 8. El error viaja en el cuerpo de la respuesta, no en el código de salida de <code>curl</code> : al descartar la salida, el purgado parecía correcto y los datos antiguos seguían en el índice.	El script de reinicio enumera los índices con <code>_cat/indices</code> , los borra por nombre y comprueba que la respuesta contenga <code>acknowledged: true</code> .

Reto y sintoma observado	Resolución
4. El estado persistente vive en cinco sitios y ninguno se limpia solo Reiniciar el flujo para probar un cambio resulto sorprendentemente difícil. Tras corregir los pipelines, los documentos mal enrutados reaparecian; tras vaciar el registro de Filebeat, volvian a entrar 2.400 transacciones antiguas con latencias de horas. El estado esta repartido en: los indices de Elasticsearch, la cola persistente de Logstash (<code>lsdata</code>), el registro de Filebeat (<code>fbdata</code>), la marca de agua del sondeo incremental y el propio fichero NDJSON. Cada uno reintroduce datos por su cuenta.	Un unico script, <code>elk/00_reiniciar_flujo.sh</code> , purga los cinco. Ademas hay que eliminar el contenedor y no solo detenerlo: Docker se niega a borrar un volumen que un contenedor todavia referencia, aunque este parado, y el fallo es silencioso.
5. Un pipeline mal configurado no impide que Logstash arranque El filtro <code>translate</code> no admite el ajuste <code>tag_on_failure</code> . Logstash arranco con normalidad, los otros cuatro pipelines funcionaron y el puerto 5044 simplemente no quedo a la escucha: Filebeat acumulaba eventos contra un destino que rechazaba la conexion. El registro tenia el motivo, pero la operacion aparentaba haber ido bien.	El script de reinicio ya no confia en el mensaje de arranque: consulta <code>/_node/pipelines</code> y compara la lista de pipelines activos con los cinco esperados, extrayendo del registro la causa si falta alguno.
6. El punto de entrada de la imagen reescribe la configuracion Al declarar <code>XPACK_MONITORING_ENABLED</code> como variable de entorno, el arranque de Logstash intenta reescribir <code>logstash.yml</code> ; el montaje es de solo lectura y el contenedor entra en un bucle de reinicio con el mensaje <code>read-only file system</code> .	El ajuste se declara dentro del propio <code>logstash.yml</code> y se retira del entorno.
7. El controlador de MySQL convierte TINYINT(1) en booleano El catalogo marcaba las 26 entidades como no vigentes, incluidas las activas con vigencia hasta el ano 9999. El controlador de MySQL entrega las columnas <code>TINYINT(1)</code> como <code>Boolean</code> — comportamiento de <code>tinyInt1isBit</code> —, de modo que la comparacion <code>[es_vigente] == 1</code> escrita en Logstash era siempre falsa. El fallo no produjo ningun error: simplemente el enriquecimiento del canal en vivo dejo de encontrar entidades.	La conversion se resolvió en la propia consulta, donde el tipo es inequivoco: <code>IF(es_vigente = 1, 'true', 'false')</code> . Logstash solo hace el <code>convert</code> final a booleano.
8. La cola interna de Filebeat marcaba la latencia del canal Resuelto el enriquecimiento, la latencia seguia en 5,4 s de mediana y 10,3 s en el percentil 99. La cifra delataba la causa: coincide con <code>queue.mem.flush.timeout</code> , que Filebeat fija en 10 s. El agente acumula 1.600 eventos antes de enviar y, si no los reúne, espera ese temporizador. A 20 eventos por segundo el umbral no se alcanza nunca, de modo que todos los lotes salian por vencimiento del reloj y no por volumen.	Se bajo el umbral a 32 eventos y el temporizador a 1 s. La latencia p99 paso de 10.257 ms a 1.142 ms sin tocar ningun otro componente. En un canal near real-time el compromiso se invierte: conviene enviar lotes pequenos a menudo antes que lotes grandes con retraso.

Reto y sintoma observado	Resolución
9. Una consulta por evento hunde la latencia El enriquecimiento se implemento primero con el filtro <code>elasticsearch</code>, que consulta el indice del catalogo una vez por evento. Cada consulta es un viaje de ida y vuelta por HTTP: el pipeline no lograba sostener 20 eventos por segundo, la cola crecia sin limite y la latencia p99 alcanzo 313 segundos. Un canal near real-time con esa latencia deja de serlo.	Una dimension de 25 filas no se consulta por red. Se materializa como diccionario y el filtro <code>translate</code> la resuelve en memoria, con recarga cada 300 s. El coste por evento pasa de una peticion de red a una busqueda en un mapa.
10. El vaciado por linea contra un montaje de macOS estrangula al generador El generador escribia el fichero NDJSON en un directorio del anfitrión montado en el contenedor, con <code>flush()</code> tras cada linea. Cada escritura sincrona sobre ese tipo de montaje cuesta milisegundos: el simulador solicitado a 20 eventos por segundo entregaba 5,6, y en una corrida se detuvo cinco minutos y medio.	El fichero se traslado a un volumen de Docker compartido entre el generador y Filebeat, con lo que el sistema de ficheros del anfitrión sale del camino critico. El <code>flush()</code> se mantiene, porque sin el Filebeat no ve las lineas.
11. La clave natural del CSV no es unica La huella de idempotencia se calculo primero sobre (fecha, entidad, provincia, canton, segmento). El resultado fueron 9.422 documentos en lugar de 9.432: diez pares de filas comparten esas cinco claves con importes distintos — en Quito coexisten un registro de USD 1,00 y otro de USD 4.614 millones —, de modo que la segunda fila sobrescribia a la primera.	La huella SHA-256 incluye tambien las cuatro medidas. Asi se descartan los duplicados exactos sin perder filas legitimas.
12. Entrega «al menos una vez» frente a idempotencia Logstash garantiza que un evento se entrega al menos una vez, no exactamente una vez. Sin identificador de documento, cada reejecucion del flujo batch anadia 176.990 documentos nuevos en lugar de actualizar los existentes.	Cada flujo define un <code>document_id</code> determinista: la clave subrogada del almacen, la clave compuesta del catalogo o la huella SHA-256 del CSV.
13. Zona horaria del controlador JDBC El controlador entrega las columnas <code>DATE</code> en la zona del servidor y Logstash las reinterpreta en UTC. Los cortes de fin de mes retrocedian un día, con lo que el saldo de mayo caía en abril.	La consulta extrae la fecha tambien como texto (<code>fecha::text</code>) y el filtro <code>date</code> fija explicitamente <code>America/Guayaquil</code>.
14. El mapeo dinamico convierte los importes en texto Si Logstash escribe antes de que exista la plantilla de indice, Elasticsearch deduce el tipo de cada campo. Los <code>NUMERIC</code> quedaban como <code>text</code> y ninguna agregacion (<code>sum</code> , <code>avg</code>) podia ejecutarse.	Las cinco plantillas se registran antes de levantar Logstash; el orden esta forzado en los scripts numerados.
15. El campo <code>tags</code> heredaba el mapeo dinamico Al no declararse en la plantilla, Elasticsearch tipaba <code>tags</code> como texto analizado. El analizador estandar parte <code>_sin_catalogo</code> en dos terminos, asi que la consulta <code>term</code> de la prueba de verificacion nunca coincidía y reportaba cero incidencias aunque las hubiera.	La plantilla declara <code>tags</code> como <code>keyword</code>. Las etiquetas de control se nombran sin guion bajo inicial.

Reto y sintoma observado	Resolución
16. La importacion de objetos de Kibana encadena migraciones antiguas El tablero se creo primero con la API de importacion. Kibana devolvía un error 500 y en su registro aparecia <code>Cannot read properties of undefined (reading 'layers')</code>: al importar, el servidor aplica la cadena historica de migraciones del tipo <code>lens</code>, y una de las antiguas espera una estructura que un objeto escrito a mano ya no tiene.	Los objetos se crean uno a uno con la API de objetos guardados (<code>POST /api/saved_objects/lens/<id></code>), que los almacena directamente en la version vigente y no ejecuta esa cadena.
17. Un solo nodo no puede alojar replicas Con la replica que Elasticsearch crea por omision, los fragmentos quedan sin asignar y el cluster permanece en estado amarillo.	Las plantillas fijan <code>number_of_replicas: 0</code>. El cluster reporta estado green con 34 fragmentos activos y 0 sin asignar.
18. Dependencia entre el flujo batch y el near real-time El enriquecimiento de cada transaccion resuelve la entidad contra el catalogo que comparte con el flujo B2. Si el diccionario no se ha materializado, las transacciones caen a <code>NO_DEFINIDO</code> y se marcan con la etiqueta <code>sin_catalogo</code>.	El script de preparacion materializa el diccionario ANTES de levantar Logstash, y la prueba V7 cuenta cuantas transacciones quedaron sin correspondencia (0 en esta corrida).
19. Permisos del fichero de configuracion montado Filebeat rechaza arrancar si su fichero de configuracion es escribible por otros usuarios. Un volumen montado desde macOS nunca cumple esa comprobacion.	Se arranca con <code>--strict.perms=false</code>, aceptable en un entorno de desarrollo contenerizado.

10. Conclusiones

1. La arquitectura integra **cinco fuentes** sobre tres tecnologias de origen y las consolida en 189.003 documentos, combinando en un mismo flujo el esquema en lote y el near real-time.
2. Los dos mecanismos de ingesta continua se implementaron de forma contrastada: el empuje mediante Filebeat alcanza una latencia p95 de 1.099 ms, mientras que el sondeo incremental por JDBC queda acotado por su intervalo de 15 segundos. La eleccion entre ambos no es de rendimiento sino de control: solo el segundo garantiza que no se pierda un registro si el consumidor esta caido.
3. El enriquecimiento del canal en vivo contra el indice que carga el lote demuestra que los dos carriles no son independientes: el esquema mixto aporta valor precisamente cuando se cruzan.
4. La mayor dificultad no fue el volumen sino los **fallos silenciosos**: un borrado rechazado que parece exitoso, una cola persistente que reproduce datos viejos y una opcion de configuracion que desactiva el aislamiento entre pipelines. Los tres se detectaron por conciliacion de conteos, no por mensajes de error, lo que justifica que la verificacion forme parte del propio flujo.

Anexo · Contenido del entregable

Ruta	Contenido
<code>docker/docker-compose.yml</code>	Nueve servicios; el perfil <code>elk</code> añade los cuatro de la pila
<code>docker/elk/logstash/pipeline/</code>	Los cinco pipelines: tres en lote y dos en near real-time
<code>docker/elk/logstash/config/</code>	Configuración global y declaración de los pipelines
<code>docker/elk/filebeat/filebeat.yml</code>	Agente de cola del canal de transacciones
<code>elk/00_reiniciar_flujo.sh</code>	Reinicio limpio de índices, cola persistente y marca de agua
<code>elk/01_configurar_elasticsearch.sh</code>	Política de ciclo de vida, cinco plantillas y flujo de datos
<code>elk/01b_diccionario_entidades.py</code>	Materializa el diccionario de enriquecimiento en memoria
<code>elk/02_simulador_transacciones.py</code>	Generador del canal near real-time
<code>elk/03_verificacion.sh</code>	Las ocho pruebas de la sección 8
<code>elk/04_kibana_objetos.sh</code>	Seis vistas de datos y el tablero de cuatro paneles
<code>elk/05_informe_elk.py</code>	Generador de este informe